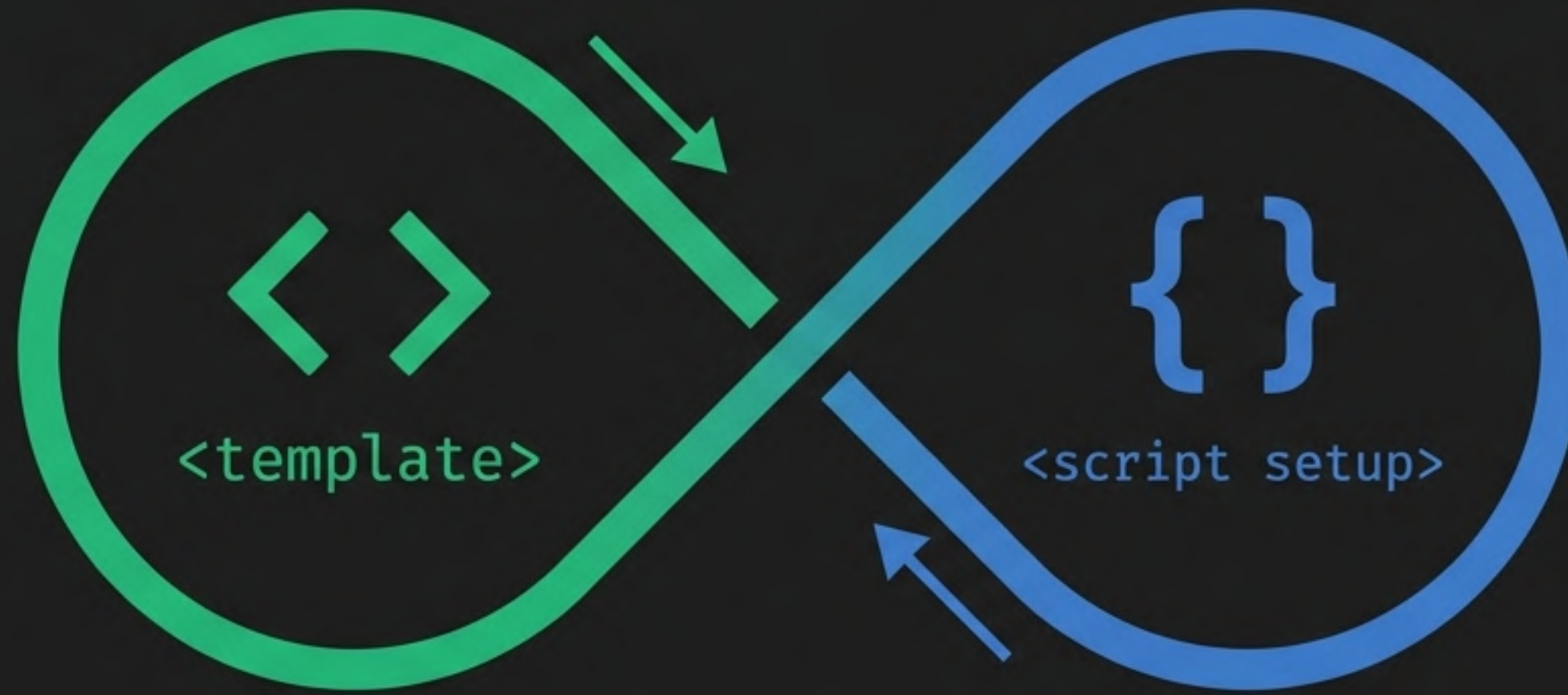




Binding de Formularios en Vue 3

v-model, value binding, componentes SFC, props, defineModel y validaciones con Composition API.

Objetivo de la clase

La Escalera de Dominio



**Comprender cómo
Vue conecta datos e
interfaz.**

(Fundamentos)



**Usar formularios
reactivos con
Composition API.**

(Estado y Lógica)



**Separar partes del
formulario en
componentes
reutilizables.**

(Arquitectura)

Binding en Vue

El binding es la sincronización entre el estado de la aplicación (datos) y la interfaz de usuario (DOM).

Unidireccional

Los datos fluyen en una sola dirección. Si el dato cambia, la vista se actualiza.



```
<p>{{ nombre }}</p>  
<input :value='nombre'>
```

Bidireccional

Los datos fluyen en ambas direcciones. Si el usuario escribe, el dato se actualiza.



```
<input v-model='nombre'>
```

v-model: two-way binding

La directiva que permite two-way binding entre un campo de formulario y una variable reactiva.

```
<script setup>
import { ref } from 'vue'

const nombre = ref('Ana')
</script>
```

```
<template>
  <input v-model="nombre" placeholder="Escribe tu nombre">
  <p>Hola, {{ nombre }}</p>
</template>
```

Cuando el usuario escribe, Vue **actualiza automáticamente** la variable reactiva vinculada con v-model.

v-model en inputs de texto y números

Los modificadores transforman la entrada del usuario antes de guardarla en el estado.

```
<input type='text'  
  v-model.trim='nombre'>
```



```
<input type='number'  
  v-model.number='edad'  
  min='0' max='120'>
```



v-model en textarea

Para textos largos, el modificador `.lazy` actualiza el dato cuando se **confirma el cambio**, en lugar de hacerlo en cada escritura.

Escribe tu biografía aquí...

50/200 caracteres

`.lazy` actualiza el dato
en el evento `change`

```
<script setup>
import { ref, computed } from 'vue'

const bio = ref('')
const contador = computed(() =>
  bio.value.length)
</script>

<template>
  <textarea v-model.lazy='bio' rows='4'>
</textarea>
  <span>{{ contador }} caracteres</span>
</template>
```


Radio buttons con v-model

En un grupo, v-model se enlaza a una única variable que guarda el **:value** de la opción seleccionada.

```
<script setup>
import { ref } from 'vue'

const nivel = ref('junior')

const opciones = [
  'junior',
  'semi senior',
  'senior'
]
</script>
```

v-for

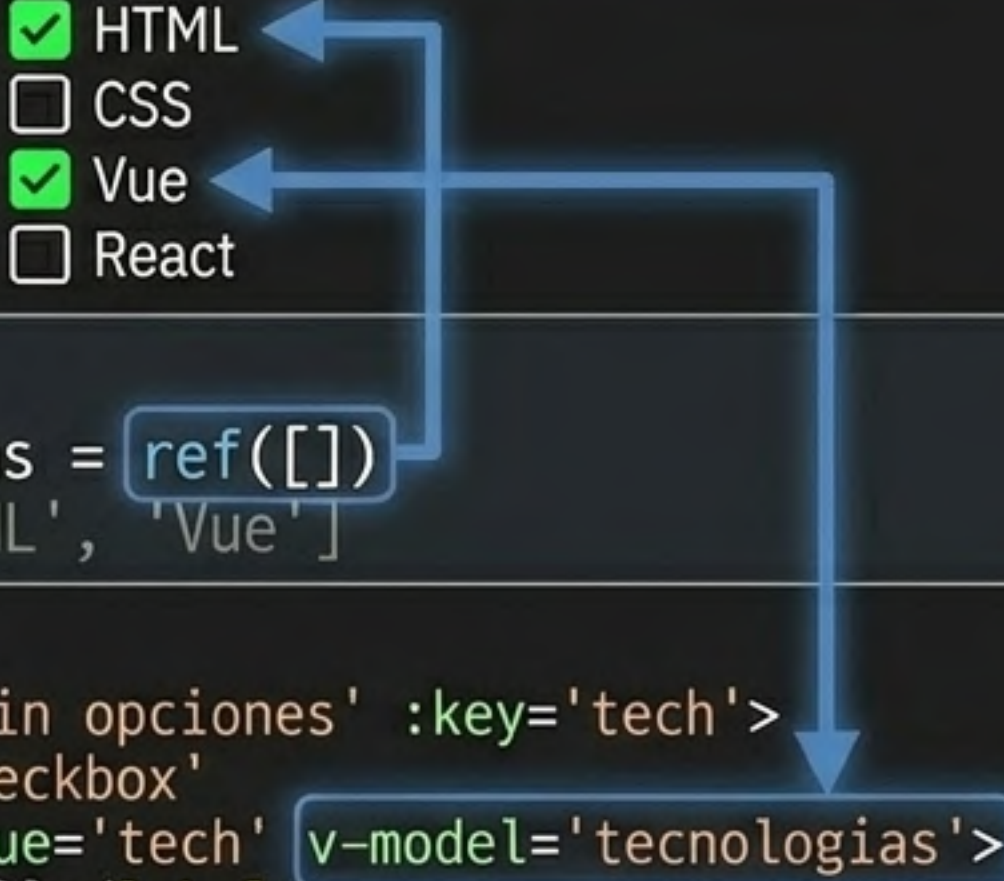
```
<template>
  <div v-for='op in opciones' :key='op'>
    <input
      type='radio'
      :value='op'
      v-model='nivel'
    >
    <label>{{ op }}</label>
  </div>

  <p>Nivel seleccionado: {{ nivel }}</p>
</template>
```

☒ junior ☐ semi senior ☐ senior

Checkbox group con v-model

Un checkbox individual guarda un booleano. Un grupo que comparte el mismo v-model guarda un Array.

Checkbox Booleano (Individual)	Checkbox Group (Múltiple)
 Acepto términos	
Código de estado <code>const aceptado = ref(true)</code>	Código de estado <code>const tecnologias = ref([])</code> <code>// Guarda: ['HTML', 'Vue']</code>
Código de interfaz <code><input type='checkbox' v-model='aceptado'></code>	Código de interfaz <code><div v-for='tech in opciones' :key='tech'></code> <code> <input type='checkbox' :value='tech' v-model='tecnologias'></code> <code> <label>{{ tech }}</label></code> <code></div></code>
Alterna entre true y false.	Recoge dinámicamente los valores (:value) de las casillas marcadas.

Select simple y select múltiple

Primero veremos un select con strings. Luego veremos cómo guardar objetos usando :value.

Un <select> estándar guarda un string. Un <select multiple> permite seleccionar varias opciones y guarda el resultado en un array.

```
<script setup>
import { ref } from 'vue'

const paises = ['Chile', 'México', 'Argentina']

const paisSeleccionado = ref('Chile')
const paisesSeleccionados = ref(['Chile', 'México'])
</script>
```

```
<template>
  <select v-model="paisSeleccionado">
    <option
      v-for="pais in paises"
      :key="pais"
      :value="pais"
    >
      {{ pais }}
    </option>
  </select>

  <select v-model="paisesSeleccionados" multiple>
    <option
      v-for="pais in paises"
      :key="pais"
      :value="pais"
    >
      {{ pais }}
    </option>
  </select>
</template>
```

A single select dropdown menu with a white background and a thin border. It contains the text "Chile" and a small downward arrow icon on the right side.A multiple select dropdown menu with a white background and a thin border. It is open, showing a list of options: "Chile" and "México". Both options are highlighted with a light gray background.

Value binding con :value

Vue permite que el valor enlazado sea un objeto complejo de JavaScript, no solo un string estático.

Comparación: `value="CL"` guarda texto. `:value="pais"` guarda el objeto completo.

Código de estado

```
<script setup>
import { ref } from 'vue'

const paises = [
  { code: 'CL', name: 'Chile' },
  { code: 'MX', name: 'México' }
]

const paisDestino = ref(null)
</script>
```

Código de interfaz

```
<template>
  <select v-model="paisDestino">
    <option disabled :value="null">
      Selecciona un país
    </option>

    <option
      v-for="pais in paises"
      :key="pais.code"
      :value="pais"
    >
      {{ pais.name }}
    </option>
  </select>

  <p>País seleccionado: {{ paisDestino }}</p>
</template>
```

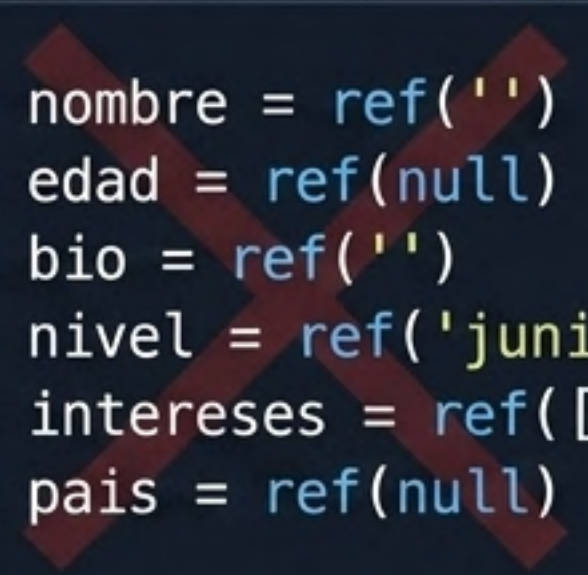
"Con `:value`, el valor enlazado puede ser una variable, un número, un booleano o incluso un objeto completo."

Estado del formulario con reactive

Para formularios grandes, es mejor agrupar los datos en un solo objeto para mantener la organización y facilitar el envío.

Antes - Micro Estado

```
const nombre = ref('')  
const edad = ref(null)  
const bio = ref('')  
const nivel = ref('junior')  
const intereses = ref([])  
const pais = ref(null)
```



Después - Macro Estado

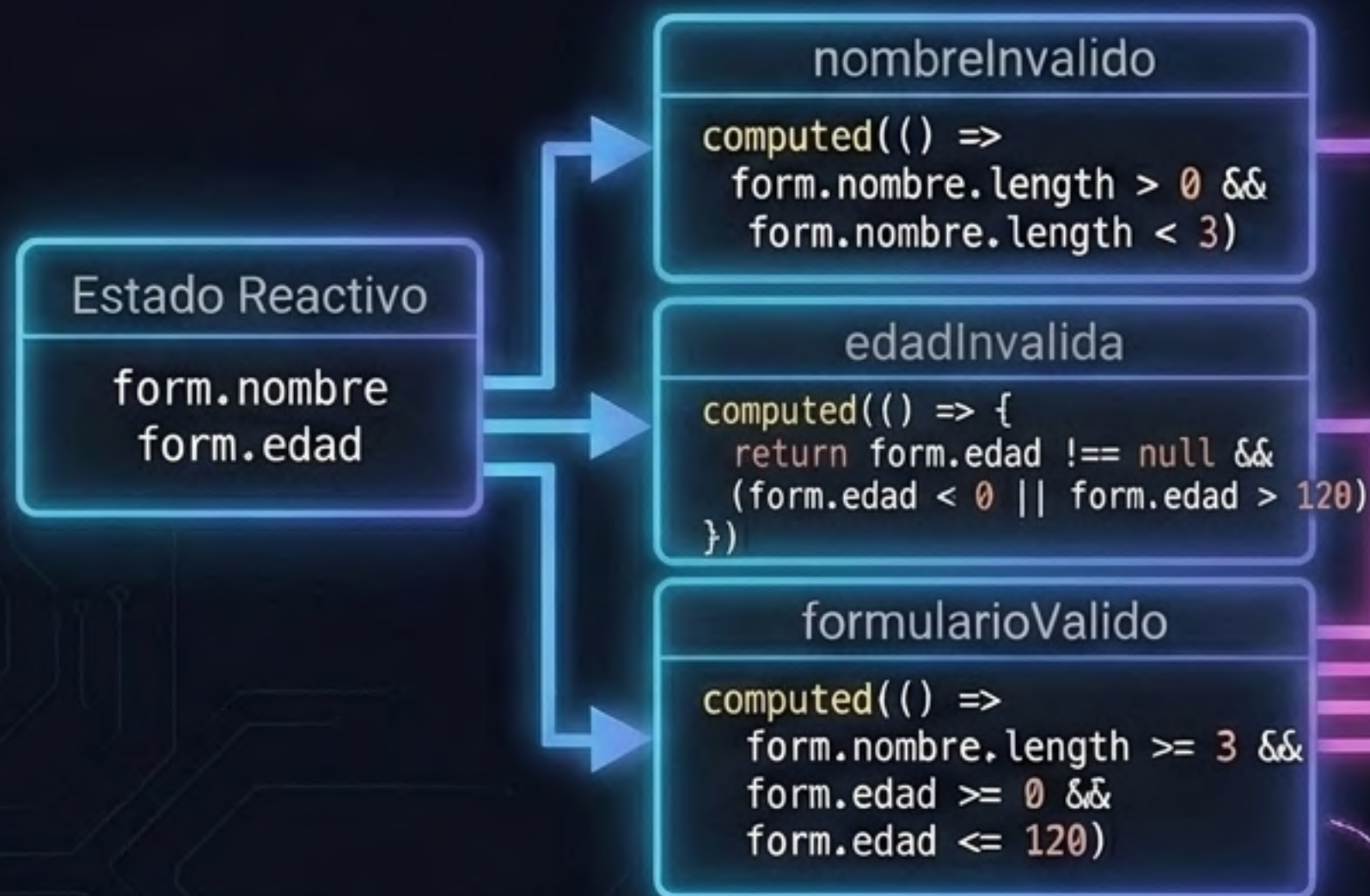
```
import { reactive } from 'vue'  
  
const form = reactive({  
  nombre: '',  
  edad: null,  
  bio: '',  
  nivel: 'junior',  
  intereses: [],  
  pais: null  
})  
  
const enviar = () => {  
  console.log(form)  
}
```

✓ Estado organizado

Validación simple con computed y :class

Usamos computed para derivar el estado de validación y bloquear o alertar en la interfaz dinámicamente.

Flujo Lógico en JavaScript



En el template no usamos `.value` con `computed`, porque Vue lo desempaqueta automáticamente.

Código de la plantilla HTML

```
<input v-model.trim="form.nombre"  
  :class="{ error: nombreInvalido }"  
  placeholder="Nombre">
```

Visualización de error

```
<input type="number"  
  v-model.number="form.edad"  
  :class="{ error: edadInvalida }"  
  placeholder="Edad">
```

Visualización de error

```
<button type='submit'  
  :disabled='!formularioValido'  
  Enviar  
</button>
```

Bloquea el envío con `:disabled` si `formularioValido` es false.

Componentes SFC y paso de props

Un Single File Component (.vue) permite agrupar template, lógica y estilos en un solo archivo. Las props permiten pasar datos del componente padre al hijo. En el hijo se leen, pero no se modifican directamente.

Estructura básica de un SFC

```
<template>
  <!-- HTML del componente -->
</template>

<script setup>
  /* lógica del componente */
</script>

<style scoped>
  /* estilos del componente */
</style>
```

Flujo de los Props

Componente Padre (App.vue)

```
<!-- App.vue -->
<template>
  <BaseInput
    label="Nombre"
    placeholder="Escribe tu nombre"
  />
</template>
```

↓ Props: label, placeholder

Componente Hijo (BaseInput.vue)

```
<!-- BaseInput.vue -->
<script setup>
  defineProps({
    label: String,
    placeholder: String
  })
</script>

<template>
  <label>{{ label }}</label>
  <input
    type="text"
    :placeholder="placeholder"
  >
</template>
```


v-model en componentes con defineModel

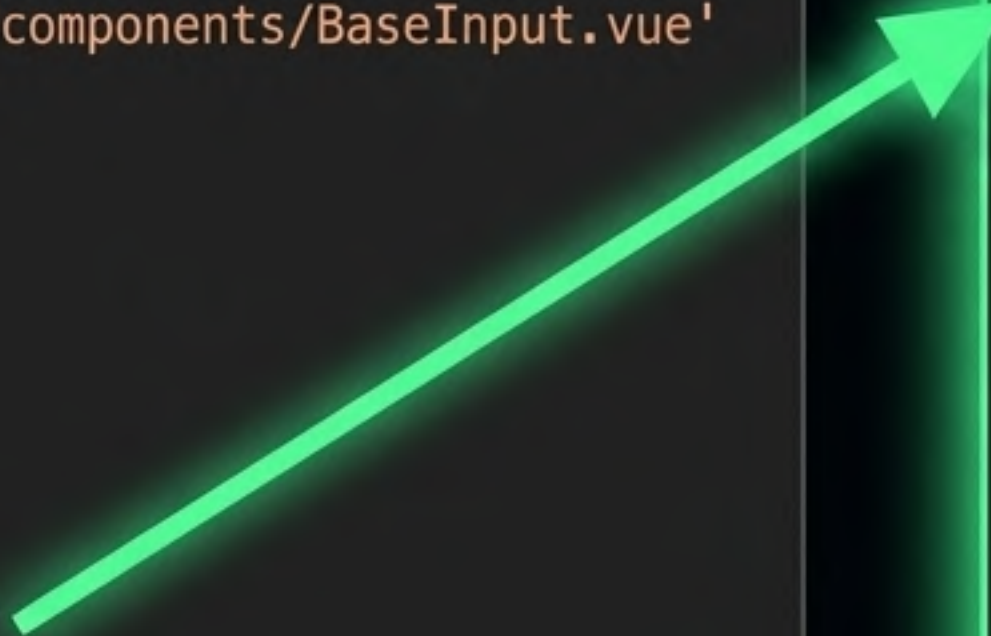
El padre entrega el valor con `v-model` y el hijo puede actualizarlo sin emitir eventos manualmente.

Componente Padre (App.vue)

```
<script setup>
import { reactive } from 'vue'
import BaseInput from './components/BaseInput.vue'

const form = reactive({
  nombre: ''
})
</script>

<template>
  <BaseInput
    v-model="form.nombre"
    label="Nombre"
  />
  <p>Nombre: {{ form.nombre }}</p>
</template>
```



Componente Hijo (BaseInput.vue)

```
<script setup>
const model = defineModel()

defineProps({
  label: String
})
</script>

<template>
  <label>{{ label }}</label>
  <input v-model="model" type="text">
</template>
```


Resumen final

Este es el conjunto de herramientas para construir formularios modernos, reutilizables y mantenibles en Vue 3.

Directivas HTML

- `v-model` (Binding bidireccional)
- `v-for` (Listas dinámicas y opciones)
- `:value` (Binding de objetos y variables)
- `@submit.prevent` (Intercepción de eventos)

Modificadores

- `.trim` (Corta espacios en blanco)
- `.number` (Transforma a tipo numérico)
- `.lazy` (Sincroniza en el evento change, no input)

Estado y Lógica

- `ref` (Estado individual, strings/arrays)
- `reactive` (Estado agrupado para todo un form)
- `computed` (Validaciones derivadas en tiempo real)
- `:class` (Renderizado dinámico de errores en UI)

Arquitectura SFC

- `.vue files` (Encapsulación de bloque)
- `defineProps` (Entrada de datos unidireccional)
- `defineModel` (Reusabilidad moderna de v-model)